

latent $\rightarrow$ hidden Stochastic / Deterministic
 (continuous $\times$ discrete) $\rightarrow$ (discrete, continuous) $\rightarrow$ (discrete, continuous)
 input $\rightarrow$ input $\rightarrow$ input
 different output $\rightarrow$ same output
 (probabilities) $\rightarrow$ at the time

variational inference $\rightarrow$ Turn inference into an optimization problem by introducing an approximate distribution
 instead of $p(z|x)$ we choose a simpler dist $q(z)$ and optimize it to be close to true posterior

inference The phase where we model data, provide and run it through model after generating train and generate/predict the test
 (to have inference is computing/approximating hidden variables given observed data $p(z|x)$) $\rightarrow$ decides pass the train and then it's applied to an output

Stochastic Gradient Variational Bayes (SGVB) estimator

Can be used for efficient approximate posterior inference in any model with continuous latent variables/parameters

Straight forward to optimize using stochastic gradient descent techniques

(IID) $\rightarrow$ independent & identically distributed
 each sample doesn't influence others
 (i.e. using one data point doesn't give any information abt other)
 All samples come from probability distribution (generated by some underlying process)



graphical model under consideration
 solid lines denote generative model $p(x|z)$ $p(x|z)$ generative model parameters
 dashed lines $\rightarrow$ variational approximation $q(z|x)$ to the intractable posterior $p(z|x)$
 variational parameters

EM - Expectation Maximization Algorithm

- $\rightarrow$ Method to estimate parameters θ when you model has hidden variables z
- $\rightarrow$ EM alternates between estimating hidden variables & updating parameters

{ GUESS & REFINE }
 loop

Monte Carlo $\rightarrow$ using random sampling to approximate integrals

if you can't compute $\int f(x)p(x)dx$ $\rightarrow$ Monte Carlo then impossible
 you approximate with samples $\rightarrow$ interpolates into sampling problems
 $\frac{1}{N} \sum_{i=1}^N f(x_i) \approx \int f(x)p(x)dx$ $\rightarrow$ Works even in higher dimensions

instead of solving integral analytically $\rightarrow$ simulate
 Average Approximate

* not n latent variables $\rightarrow$

z lives in a continuous high dimensional space
 we must consider all values of z
 no the difficulty is integration over a huge continuous space

* We are not trying to find which set of latent variables generated the sample

We are trying to compute $p(z|x)$

this is a probability distribution over latent space

stochastic variational inference means

we optimize the variational objective using stochastic optimization (random sampling / mini batches)

i.e. instead of computing expectations over the entire dataset (or full distribution) $\rightarrow$ we use

sample data point & latent variables to estimate gradient

Inference $\neq$ any compilation

$\rightarrow$ computing unknown quantities conditioned on known objectives

Ancestral sampling - sampling variables in the order of the directed graph parents $\rightarrow$ children

eg $z \rightarrow x$ (directed model)

$$p(x, z) = p(z)p(x|z) \quad (joint)$$

Distributions
 (i) Discrete distributions (when outcomes are countable)
 $\rightarrow$ Bernoulli $\rightarrow$ one trial $\rightarrow$ probability of success
 $\rightarrow$ Binomial distribution $\rightarrow$ No. of successes in N trials
 $\rightarrow$ Poisson distribution $\rightarrow$ Counts in a time/space interval ($\lambda \rightarrow$ avg rate)
 (ii) Continuous distributions (when outcomes are real numbers)
 $\rightarrow$ Normal / Gaussian distribution $\rightarrow$ famous bell curve (σ^2 variance, μ mean)
 $\rightarrow$ Uniform distribution $\rightarrow$ All values equally likely
 $\rightarrow$ Exponential distribution $\rightarrow$ waiting time between events (rate) (Memoryless property)
 $\rightarrow$ Gamma distribution $\rightarrow$ Generalization of exponential (multiple waits)
 $\rightarrow$ Beta $\rightarrow$ distribution of probabilities (range 0 to 1) (α, β parameters that control shape)
 Regressive $\rightarrow$ Bernoulli / Binomial $\rightarrow$ as r/o processes
 Poisson / Exponential $\rightarrow$ event timing / counts
 Normal $\rightarrow$ noise & natural variation
 Beta $\rightarrow$ uncertainty about probabilities
 Gamma $\rightarrow$ accumulated waiting

Rep of main probability functions

(i) PMF $\rightarrow$ probability mass function (discrete)

$\rightarrow$ and for discrete random variable

gives probability of exact values $P(X=x)$ (unlike discrete)
 $\sum_x P(X=x) = 1$
 Bernoulli & binomial (processes)
 Geometric

(ii) PDF - Probability Density function (continuous)

used for continuous random variables

You cannot see the probability of an exact value

* Probability Over an Interval

$$P(a \leq X \leq b) = \int_a^b f(x)dx$$

Key property $\rightarrow \int_{-\infty}^{\infty} f(x)dx = 1$
 Gaussian, exponential, Beta, gamma, Uniform

A PDF is a density not probability

Height of the curve is not probability $\rightarrow$ Area is

(iii) Cumulative Distribution Function (CDF)

works for both discrete & continuous variables

$$F(x) = P(X \leq x)$$

it accumulates probability upto x

proportion $\rightarrow$ always increasing

ranges from 0 to 1

right continuous

for continuous variables
 $f(x) = d(F(x))$
 or
 PDF = derivative of CDF
 CDF = integral of PDF

Discrete $\rightarrow$ Probability $\rightarrow$ Height

Continuous $\rightarrow$ Probability $\rightarrow$ Area

Ancestral sampling - sampling variables in the order of the directed graph parents $\rightarrow$ children

eg $z \rightarrow x$ (directed model)
 $p(x, z) = p(z) p(x|z)$ (joint)

ancestral sampling $\Rightarrow$
 1 sample $z \rightarrow p(z)$ you always sample
 2 sample $x \rightarrow p(x|z)$ parents first, then children

Directed probabilistic models define a generative model $\rightarrow$ ancestral sampling follows it to generate data

in probabilistic models you usually have

latent variables $\rightarrow z$ (you don't see)

Global parameters $\rightarrow \theta$

eg VAE style thinking $p(z|\theta)$

$\theta \rightarrow$ changes per sample
 & defines the model itself (its parameters etc)

Maximum likelihood on global parameters
 goal (how θ that seen observed data most probable)

Mathematically $\theta^* = \arg \max_{\theta} p(\theta)$

or for distinct $\theta = \arg \max_{\theta} \sum \log p(\theta)$ treat θ as unknown but find best the value that best explains data
 no prior assumption on θ

$\rightarrow$ choose param that explain the data best

Discrete $\rightarrow$ Probability $\rightarrow$ Height
 Continuous $\rightarrow$ Probability $\rightarrow$ Area

PDF = cumulative of POF
 CDF = integral of PDF

Maximum A posteriori (MAP) $\Rightarrow$ global parameter

is assume a prior $p(\theta)$
 goal $\theta^* = \arg \max_{\theta} p(\theta|x)$

using bayes $\theta^* = \arg \max_{\theta} p(x|\theta)p(\theta)$

eg log prior $\log p(x|\theta) + \log p(\theta)$

$\rightarrow$ choose params that explain the data well and look plausible under prior beliefs

Because we are not inferring latent variables here
 we are optimising the shared parameters that affect the whole model

what "causality" means here

In directed probabilistic models "causality" is a generative direction

$z \rightarrow x$ first sample z then generate x

$\rightarrow$ is a modeling assumption, not proof that z truly causes x in reality

"direction" in ML paper usually means probabilistic dependence, sampling order

structured causal decomposition

without structure $p(x, z)$ is one giant object - exponential complexity
 because variables are generated through smaller conditions/priors

Instead of learning $p(x, z)$

assume structure like $p(x, z) = p(z|x) p(x|z)$

so we decompose the joint using directed dependencies

Structure = graph

Decomposition = factorization into local conditionals

Why this helps?

Because each piece is simpler learn many small components instead of one impossible joint

In VAE $p(x, z) = p(z) p(x|z)$

complex data x comes from simple latent causes z

Directed $\neq$ true causality

Modelling causality $\rightarrow$ how we choose to generate data

Real causality $\rightarrow$ physics/intentions etc

Basically standard causal decomposition is representing a complex joint distribution as

a product of simple conditional distributions arranged in a directed generative order

Directed probabilistic Models -

Also called directed graphical model or bayesian network

A way to represent joint probability distributions using a directed graph

$\rightarrow$ represent random variables as

Nodes $\rightarrow$ random variables

Directed edges $\rightarrow$ probabilistic dependence (cause $\rightarrow$ effect)

Key idea - The joint probability distribution factorizes according to the graph structure

$p(x_1, x_2) = \prod p(x_i | \text{parents}(x_i))$ Every variable is conditioned only on its parent nodes

Representation Instead of storing one huge joint distribution

$p(x_1, x_2)$
 we represent it as local conditional distributions

eg if the graph is $z \rightarrow x$

then $p(x, z) = p(z) p(x|z)$

This is exactly the structure used in VAE "latent variables" "observed data"

Objective - The goal is to model $p(x)$

But directed models introduce hidden variables

$p(x) = \int p(x|z) p(z) dz$

so the model defines a data generation process

1 sample latent causes z

2 generate data x from it

Information flow

flow direction $z \rightarrow x$

information originates in latent variables

Data is generated from compressed causes $\rightarrow$ this is why directed models are often called generative models

Constraint - why? problem $\rightarrow$ directly modelling high dimensional distributions

solution $\rightarrow$ Assume structured causal decomposition

"model small conditionals instead of one massive dist"

Removal (what breaks?)

1) if you remove directionality $\rightarrow$ you get undirected models (Boltzmann machines)

* model small conditionals instead of one massive distⁿ

Removal (what breaks?)

- 1) if you remove directionality $\rightarrow$ you get undirected models (Boltzmann machines)
 - no explicit sampling order
 - needs normalization (partition functions)

Directed models give $\rightarrow$ explicit sampling process
easier: oncentra/sampling

7) Replacement

Type	example	key difference
Directed	Bayesian networks, GANs/generators, VAEs	
Undirected	Boltzmann machines, Symmetric interactions	
Flow-based	Normalizing flows, invertible deterministic transforms	

Gradient loss -

Learning Maximizes likelihood

$$\nabla_{\theta} \log P(x)$$

gradient flow through
conditional distributions $P(x|z)$
latent structure

Problem (see VAE motivation)
Integrating over latent variables using gradient flow
This directly leads to variational inference

if Overall, you can reparameterize it as A factored joint distribution where the graph structure defines conditional dependencies and a generative sampling process
 $P(x, z) = p(z)P(x|z)$

Variational inference $\rightarrow$ approximation method (usually for latent variables)

ML/MAPF $\rightarrow$ objective for parameter estimation

problem scenario consider dataset $X = \{x^{(i)}\}_{i=1}^N$ consisting of N i.i.d samples of continuous/ discrete variable x .

Assume data generated by a random process, modeling an unobserved continuous random variable z

two steps (i) a value $z^{(i)}$ is generated from some prior distribution $P_0^*(z)$

(ii) a value $x^{(i)}$ is generated from some conditional distribution $P_0^*(x|z)$

We assume $P_0^*(z) \in \mathcal{P}_0$ and $P_0^*(x|z) \in \mathcal{P}_0(x|z)$ come from parametric families of distributions $\mathcal{P}_0(z)$ and $\mathcal{P}_0(x|z)$ and thus PDF's are differentiable almost everywhere w.r.t both z and x
 prior likelihood All dist's in the family share

unfortunately a lot of this process is hidden from our view
same shape but differ based on the specific values of a few fixed parameters

A general algorithm that can
infer efficiently in case of

The values θ^* & latent variables z are unknown to us

1) Intractability the case where the integral of marginal likelihood

$$p_0(x) = \int p_0(x|z) p_0(z) dz \text{ is intractable (no we cannot evaluate/differentiate the marginal likelihood)}$$

where the posterior density $p_0(z|x) = \frac{p_0(x|z)p_0(z)}{p_0(x)}$ is intractable (no EM alg out to us)

and where the required integrals for any reasonable mean-field VB algorithm are also intractable

These intractabilities are quite common and appear in cases of moderately complicated likelihood function $p_0(x|z)$ eg neural nets with non-linear hidden layers

ii) A large dataset we have no such data that batch optimization is too costly

we would like to make parameter updates using small mini-batches or even single datapoints

Sampling based solutions eg Monte Carlo EM would be too slow since it requires a typically expensive sampling loop per datapoint

it's technical optimization requirement
They assume $p(z)$, $p(x|z)$ are differentiable w.r.t θ & z

Why $w \neq \theta$?

because learning requires gradients $\nabla_{\theta} \log p(x)$
No differentiability no gradient based optimization

Why $w \neq z$?

because they will optimize expectations modeling z more gradients through latent sampling
if $p(x|z)$ isn't smooth, no gradient based learning becomes

True posterior density?

$p_0(z|x)$ {true posterior} true because it is the mathematically correct Bayesian posterior implied by the model using Bayes $p_0(z|x) = \frac{p_0(x|z)p_0(z)}{p_0(x)}$

they emphasize this as true because they later introduce $q_p(z|x)$ {an approximation}

true posterior $\rightarrow$ exact but intractable

Variational posterior $\rightarrow$ approximate but tractable

latent variables as unknown, posterior over them tractable

1.) A large dataset we have no much data that batch optimization is too costly

we would like to make parameter updates using small mini-batches or even single datapoints

Sampling based solutions of Monte Carlo (EM) would be too slow since it requires a typically expensive sampling loop per datapoint

↳ they propose solution to above

(i) efficient approximate MCMC estimation for the parameters Θ They can be efficient themselves. They allow us to mimic the hidden random process and generate artificial data that resembles the real data

(ii) efficient approximate posterior inference of the latent variable z given an observed value x for a choice of param Θ

(iii) efficient approximate marginal inference over the variable x This allows us to perform all kinds of inference tasks where a prior over x is required

it is about the data (x) itself with regard for specific hidden states

↳ if data $x \in \mathcal{X}$ & latent variables $z \in \mathcal{Z}$

the Marginal likelihood $p(x) = \int p(x, z) dz$

posterior inference asks given image of a cat what is most likely hidden variable

marginal inference asks overall how likely it is that my model would generate this image of a cat

They introduce a recognition model $q_\phi(z|x)$ as approximation to the intractable true posterior $p(z|x)$

a method for learning the recognition model parameters ϕ jointly with generative model parameters Θ

→ refer recognition model as probabilistic encoder

$q_\phi(z|x)$

since given a datapoint x it produces distribution over the possible values of z from which the datapoint x could have been generated

→ refer $p_\Theta(x)$ as a probabilistic decoder, since given a code z it produces distribution over the possible corresponding values of x

Decoder/generative model $p_\Theta(x|z)$

Posterior $p_\Theta(z|x)$

Marginal likelihood (evidence) $p_\Theta(x)$

conditional generation from latent code

The Variational bound

The marginal likelihood is composed of a sum over the marginal likelihoods of individual datapoints $\log p(x^{(1)} \dots x^{(n)}) = \sum_{i=1}^n \log p_\Theta(x^{(i)})$

it can be rewritten as

$$\log p_\Theta(x^{(i)}) = D_{KL}(q_\phi(z|x^{(i)}) || p_\Theta(z|x^{(i)})) + \underbrace{\mathbb{E}_{q_\phi(z|x^{(i)})} [\log p_\Theta(x^{(i)})]}_{\text{KL divergence of approximate from the true posterior}}$$

not describing replacement
it is an exact decomposition identity
 $\log p_\Theta(x) = KL(q_\phi(z|x) || p_\Theta(z|x)) + \mathbb{E}_{q_\phi(z|x)} [\log p_\Theta(x)]$
log likelihood = KL qgp + lower bound

(variational) lower bound on the marginal likelihood of datapoint i

it can be rewritten as

$$\log p_\Theta(x^{(i)}) \geq \mathcal{L}(\Theta, \phi, x^{(i)}) = \mathbb{E}_{q_\phi(z|x)} [-\log q_\phi(z|x) + \log p_\Theta(x, z)]$$

$$\text{which can also be rewritten as } \mathcal{L}(\Theta, \phi, x^{(i)}) = -D_{KL}(q_\phi(z|x^{(i)}) || p_\Theta(z)) + \mathbb{E}_{q_\phi(z|x^{(i)})} [\log p_\Theta(x^{(i)}|z)]$$

we want to differentiate & optimize this w.r.t both ϕ & Θ

However gradient of lower bound w.r.t ϕ is problematic

The SGLB estimator & AEVB algorithm

for a chosen approximate posterior $q_\phi(z|x)$ under certain mild conditions we can reparameterize the random variable $\tilde{z} = q_\phi(z|x)$ using a differentiable transformation $g_\phi(\epsilon; x)$ of an auxiliary noise variable ϵ

$$\tilde{z} = g_\phi(\epsilon; x) \text{ with } \epsilon \sim p(\epsilon) \rightarrow (4)$$

we can now form Monte Carlo estimates of expectations of some function $f(z)$ w.r.t $q_\phi(z|x)$ as follows

$$\mathbb{E}_{q_\phi(z|x^{(i)})} [f(z)] = \mathbb{E}_{p(\epsilon)} [f(g_\phi(\epsilon; x^{(i)}))] \approx \frac{1}{L} \sum_{l=1}^L f(g_\phi(\epsilon^{(l)}; x^{(i)})) \text{ where } \epsilon^{(l)} \sim p(\epsilon) \rightarrow (5)$$

We can apply this technique to the variational lower bound yielding our generic SGLB estimator $\mathcal{L}^A(\Theta, \phi, x^{(i)}) \approx \mathcal{L}(\Theta, \phi, x^{(i)})$

$$\mathcal{L}^A(\Theta, \phi, x^{(i)}) = \frac{1}{L} \sum_{l=1}^L \log p_\Theta(x^{(i)} | z^{(l,i)}) - \log q_\phi(z^{(l,i)} | x^{(i)}) \rightarrow (6)$$

where $z^{(l,i)} = g_\phi(\epsilon^{(l,i)}; x^{(i)})$ and $\epsilon^{(l,i)} \sim p(\epsilon)$

$$\mathcal{L}(\theta, \phi; x^{(1)}) = -D_{KL}(q_{\theta, \phi}(z|x^{(1)}) \| p_{\theta}(z)) + \frac{1}{L} \sum_{l=1}^L (\log p_{\theta}(x^{(l)} | z^{(l)}))$$

where $z^{(1)}, \dots, z^{(L)} \sim q_{\theta, \phi}(z|x^{(1)})$

indicator of using likelihood lower bound of the full dataset, based on minibatches

$$\mathcal{L}(\theta, \phi; X) \approx \mathcal{L}^M(\theta, \phi; X^M) = \frac{N}{M} \sum_{i=1}^M \mathcal{L}(\theta, \phi; x^{(i)})$$

$X^M = \{x^{(1)}, \dots, x^{(M)}\}$ is a randomly drawn sample of M datapoints from the full dataset X with N datapoints

→ the reparameterizing trick

we use an alternative method for generating

$$z_{\theta, \phi}(x)$$

let z be a continuous random variable

$z \sim q_{\theta, \phi}(z|x)$ some conditional distribution

it is often possible to express the random variable z as a noise variable $z = g_{\theta, \phi}(z, \epsilon)$

ϵ as an auxiliary variable with independent marginal $p(\epsilon)$

$g_{\theta, \phi}(\cdot)$ is some vector-valued function parametrized by θ, ϕ

reparameterization is useful since it can be used to rewrite an expectation $E_{z \sim q_{\theta, \phi}(z|x)}$

such that Monte Carlo estimate of the expectation is $\frac{1}{M} \sum_{i=1}^M g_{\theta, \phi}(z^{(i)}, \epsilon^{(i)})$

Proof Given the deterministic mapping $z = g_{\theta, \phi}(z, \epsilon)$

we have $q_{\theta, \phi}(z|x) \prod_i d\epsilon_i = p(\epsilon) \prod_i d\epsilon_i$

$$\text{Therefore } \int g_{\theta, \phi}(z|x) q_{\theta, \phi}(z|x) dz = \int p(\epsilon) g_{\theta, \phi}(g_{\theta, \phi}(z, \epsilon)|x) d\epsilon$$

$$\int g_{\theta, \phi}(z|x) p(z) dz = \int \frac{1}{L} \sum_{l=1}^L p(g_{\theta, \phi}(z, \epsilon^{(l)})) d\epsilon \sim p(\epsilon)$$

eg of a univariate gaussian case let $z \sim p(z) = \mathcal{N}(z; \mu, \sigma^2)$

a valid reparameterizing here would be

$$z = \mu + \sigma \epsilon \quad \text{noise variable } \epsilon \sim \mathcal{N}(0, 1)$$

$$E_{z \sim \mathcal{N}(\mu, \sigma^2)}[f(z)] = E_{\epsilon \sim \mathcal{N}(0, 1)}[f(\mu + \sigma \epsilon)] \approx \frac{1}{L} \sum_{l=1}^L f(\mu + \sigma \epsilon^{(l)}) \quad \epsilon^{(l)} \sim \mathcal{N}(0, 1)$$

the basic approaches to choose $g_{\theta, \phi}(\cdot)$ for $g_{\theta, \phi}(z|x)$ are

(1) Tractable univariate Cumulative Distribution Function In this case, let $\epsilon \sim \mathcal{N}(0, 1)$ and let $g_{\theta, \phi}(z|x)$ be the inverse CDF of $q_{\theta, \phi}(z|x)$ eg exponential, cauchy, g_1 etc distributions

(2) Analogue to the Gaussian example, for any "location-scale" family of distributions we can choose the standard distribution (with location = scale = 1) as the variable ϵ

and $g_{\theta, \phi}(\cdot)$ = location + scale eg $\epsilon \sim \text{gaussian}$ distributions

(3) Composition it is often possible to express random variables as different transformations of auxiliary variables eg log-normal, gamma, chi-squared

$\theta, \phi \leftarrow$ initialize & start with random model parameters

Sample a minibatch X^M

Sample noise $\epsilon \sim p(\epsilon)$ (if it is not learned, just random)

Generate latent variables (reparameterization)

$$z = g_{\theta, \phi}(z, \epsilon)$$

for each data point $\rightarrow$ encode computer parameter (mean, std etc)

noise gets transformed into latent sample

compute ELBO (our objective)

KL term, reconstruction term This gives a rough box estimate for the minibatch

Compute gradients

$$\text{Backpropagation } g_{\theta, \phi} \leftarrow \nabla_{\theta, \phi}$$

gradients flow through decoder, encoder, reparameterization, transform

Update parameters

θ, ϕ Move parameters in direction that improves ELBO

Repeat & then return θ, ϕ after convergence (loss stops improving)

$X \rightarrow$ encoder $\rightarrow z$

$z \rightarrow$ decoder $\rightarrow$ reconstruction $\hat{x}$

compute ELBO

backprop update

minibatch sampling noise sampling $\rightarrow$ introduce randomness

eg Variational Auto Encoder

model set from probabilistic encoder $q_{\theta}(z|x) \rightarrow$ the approximation to the true posterior of the generative model $p_{\theta}(z|x)$ where θ, ϕ are optimized by AEB

$p_{\theta}(z) = \mathcal{N}(0, I)$ centered isotropic multivariate gaussian

$p_{\theta}(z|x)$ multivariate gaussian (a mix of multivariate) or Bernoulli (in case of binary)

Latent parameters come MLP (a fully connected NN $\rightarrow$ it's a single hidden layer)

True posterior $p_{\theta}(z|x)$ is intractable

$\rightarrow$ we assume it takes on approximate gaussian with an approximately diagonal covariance

we let $q_{\theta}(z|x)$ be a multivariate gaussian with a diagonal covariance structure

$$\log q_{\theta}(z|x) = \log \mathcal{N}(z; \mu^{(1)}, \sigma^{2(1)} I)$$

are outputs of encoding MLP i.e non linear function of datapoint $x^{(1)}$ & variational parameters

$\rightarrow$ sample from posterior (reparameterization)

$$z^{(1), 1} \sim q_{\theta}(z|x^{(1)})$$

$$z^{(1), 1} = g_{\theta}(z^{(1), 1}, \epsilon^{(1)})$$

$$= \mu^{(1)} + \sigma^{(1)} \epsilon^{(1)} \quad \epsilon^{(1)} \sim \mathcal{N}(0, I) \quad \text{we sample an element noise product}$$

$q(\epsilon)$

The distributions chosen prior $p(z) \sim \mathcal{N}(0, I)$

Approximate posterior

$$q_{\theta}(z|x) \approx \mathcal{N}(\mu, \sigma^2 I)$$

↳ sample from posterior (reparameterization)

$$z^{(i,1)} \sim q_{\phi}(z|x^{(i)})$$

$$z^{(i,1)} = g_{\phi}(x^{(i)}, u^{(i)})$$

$$= \mu^{(i)} + \sigma^{(i)} \epsilon^{(i)} \quad \epsilon^{(i)} \sim N(0, I) \quad \text{with } 0 \text{ we simplify and almost use product}$$

in this model $p_{\theta}(z)$ (prior) and $q_{\phi}(z|x)$ are gaussian

we use eq (7) $\mathcal{L}(\theta, \phi; \beta, x^{(i)}) = -D_{KL}(q_{\phi}(z|x^{(i)}) || p_{\theta}(z)) + \frac{1}{L} \sum_{i=1}^L (\log p_{\theta}(x^{(i)} | z^{(i,1)}))$

$$z^{(i,1)} \sim g_{\phi}(x^{(i)}, u^{(i)}) \text{ and } \epsilon^{(i)} \sim p(\epsilon)$$

$$\mathcal{L}(\theta, \phi; x^{(i)}) \approx \frac{1}{2} \sum_{j=1}^J \left(L + \log \left(\frac{\sigma_j^{(i,1)}}{\sigma_j^{(i)}} \right) - (u_j^{(i,1)})^2 - (\sigma_j^{(i,1)})^2 \right) + \frac{1}{L} \sum_{i=1}^L \log p_{\theta}(x^{(i)} | z^{(i,1)})$$

$$z^{(i,1)} = \mu^{(i)} + \sigma^{(i)} \epsilon^{(i)} \text{ and } \epsilon^{(i)} \sim N(0, I)$$

Related work

1. Variational latent variable learning methods

Variational autoencoder

↳ Also uses a recognition model (like encoder/decoder)

problem optimizes two separate objectives, not a true likelihood bound

VAE improves this by optimizing a single variational objective

(2) Connection to stochastic variational inference

← previous work studied

stochastic gradient estimators

Variance reduction techniques

This paper introduces a clean reparameterization trick to reduce gradient variance

(3) Connection to probabilistic linear models

VAE linear probabilistic model ↔ autoencoders

eg PCA can be seen as a special case of a linear gaussian latent variable model

meaning VAE generalizes classical latent variable models using neural networks

(4) Connection to autoencoders

Traditional autoencoders ↔ reconstruction error only

VAE adds KL regularization

↳ probabilistic encoder with penalized regularization

(5) Relation to other regularized autoencoders

earlier approaches - denoising, sparse, contractive encoders

These use heuristic regularizers

VAE gets regularization naturally through ELBO

(6) VAE provides general, scalable alternatives to Boltzmann machines, Generative stochastic Nets, DARN models

VAEs unify Bayesian learning & Autoencoders

reparameterization enables efficient stochastic gradient training

ELBO gives a principled objective instead of heuristic reconstruction losses

VAE's sit between classical probabilistic latent variable models (PCA, Variational inference) and modern neural autoencoders

eq (9)

The distributions chosen

prior $p(z) = N(0, I)$

Approximate posterior

$$q_{\phi}(z|x) = N(\mu, \sigma^2 I)$$

KLDivergence between two gaussian has a closed form solution

$$\frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - \log(\sigma_j^2) - 1) \rightarrow \text{comes from gaussian KL formula}$$

! ? Because z is multivariate vector

diagonal covariance $\Rightarrow$ each dimension independent

so we compute KL per dimension & sum

↳ approximate latent dimensional index

MLEBD we have -KL

so reparam $\Rightarrow \frac{1}{2} \sum_j (1 - \log(\sigma_j^2)) - \mu_j^2 - \sigma_j^2$

second part of the equation

reconstruction remains $\frac{1}{L} \sum_{i=1}^L \log p_{\theta}(x^{(i)} | z^{(i,1)})$

$$z^{(i,1)} = \mu + \sigma \epsilon^{(i)}$$

entropy term
variance penalty
gradient term (distance of means)

Goal $\rightarrow$ to write sampling as deterministic function + sample noise

Approach 1 - Inverse CDF (first approach)

$\epsilon \sim U(0,1)$ I = identity matrix

A multivariate uniform distribution with independent components

idea if a distribution has an inverse CDF

sample uniform noise & pass it through CDF

$\epsilon \sim U(0,1)$

but $\epsilon^{(i)}$ be inverse CDF

$$z = F^{-1}(\epsilon)$$

produces samples from the desired distribution

Intuition $\rightarrow$ uniform noise is easy

inverse CDF technique d into target distribution

Why useful for reparameterization?

if F^{-1} is differentiable gradients flow through it

Approach (2) Location scale family

uniform $z = \text{location} + \text{scale} * \epsilon$

where ϵ comes from standard distribution

Example (Gaussian)

Standard noise $\epsilon \sim N(0,1)$

General gaussian $z = \mu + \sigma \epsilon$ μ shifts $\rightarrow$ location

σ stretches $\rightarrow$ scale

Any distribution that can be created by shifting and scaling a standard one belongs here

why this is easy?

randomness is additive

parameters are differentiable

Approach 3 - Composition

distributions are not around location scale

‡ ELBO contains a KL term that can be mitigated analytically

When $p_\theta(z) = \mathcal{N}(z; \mu, \sigma^2)$ and $q_\phi(z|x^{(i)})$ are gaussian

‡ demonstrate multiply of z

μ, σ are mean & std of z

μ_j, σ_j^2 denote j th element of these vectors

$$\int q_\theta(z) \log p(z) dz = \int \mathcal{N}(z; \mu, \sigma^2) \log \mathcal{N}(z; \mu, \sigma^2) dz$$

$$= -\frac{1}{2} \log(1/\pi) - \frac{1}{2} \sum_{j=1}^D (\mu_j^2 + \sigma_j^2)$$

And

$$\int q_\phi(z) \log q_\phi(z) dz = \int \mathcal{N}(z; \mu, \sigma^2) \log \mathcal{N}(z; \mu, \sigma^2) dz$$

$$= -\frac{1}{2} \log(1/\pi) - \frac{1}{2} \sum_{j=1}^D (\mu_j^2 + \log \sigma_j^2)$$

$$-D_{KL}(q_\phi(z) || p_\theta(z)) = \int q_\phi(z) (\log p_\theta(z) - \log q_\phi(z)) dz$$

$$= \frac{1}{2} \sum_{j=1}^D (1 + \log(\sigma_j^2) - (\mu_j^2 + \log \sigma_j^2))$$

Encoder $\rightarrow$ MLP with gaussian output

Binary MLP as decoder $\rightarrow p_\theta(x|z)$ a multivariate Bernoulli whose probabilities are computed from z with a MLP

$$\log P(x|z) = \sum_{i=1}^D x_i \log y_i + (1-x_i) \log(1-y_i)$$

$$y = \text{tanh}(w_2 z + b_2)$$

tanh = element-wise sigmoid activation function $\Theta = \{w_1, w_2, b_1, b_2\}$ param of MLP

Gaussian MLP as encoder & decoder

A multivariate gaussian with diagonal covariance structure

$$\log p(x|z) = \log \mathcal{N}(x; \mu, \sigma^2 I)$$

$$\text{where } \mu = w_1 z + b_1$$

$$\log \sigma^{-2} = w_2 z + b_2$$

$$b_2 = \text{tanh}(w_3 z + b_3)$$

$\{w_1, w_2, w_3, b_1, b_2, b_3\}$ are wts & biases of MLP & prob of 0 when used as decoder

When this network is used as an encoder $q_\phi(z|x)$ then $z \in x$ are mapped & wts & biases are variational parameters

parameters are differentiable

Approach 3: Composition

distributions are not sample location scale

relax Build complex distributions by combining simpler ones

eg log-normal $\in \mathcal{N}(\mu, \sigma^2)$

Transform $z = \exp(z)$

now z follows log-normal

eg z Gamma/diagonal

often sums or ratios of simple random variables

each step differentiable $\Rightarrow$ whole thing differentiable

instead of one sample transform $z \sim q(z)$ we do $z = q_\phi(q_\theta(z))$

Terminology: univariate = one dimensional random variable

multivariate = vector

multivariate gaussian: Gaussian over vectors $x \in \mathbb{R}^D$ instead of a single value

Centred: Mean = zero $\mu = 0$

Isotropic: Same variance in all directions

Covariance matrix $\Sigma = I$

Neat: distribution is perfectly symmetric

no direction preferred

Diagonal co-variance: each dimension of the data is treated as independent of the other